

MODULE 4 · NETWORKING

Pod-to-Pod Connectivity

Every Pod gets a routable IP — reachable directly, no NAT, no matter the node

Before Services, there's just Pods talking to Pods

Kubernetes requires a **flat, routable network**: every Pod gets its own IP, and any Pod can reach any other Pod's IP directly — no NAT, regardless of which node either one landed on.

That's the foundation everything else in networking sits on. Before reaching for a Service, you should know what already works with nothing but a Pod IP.

BY THE END YOU CAN

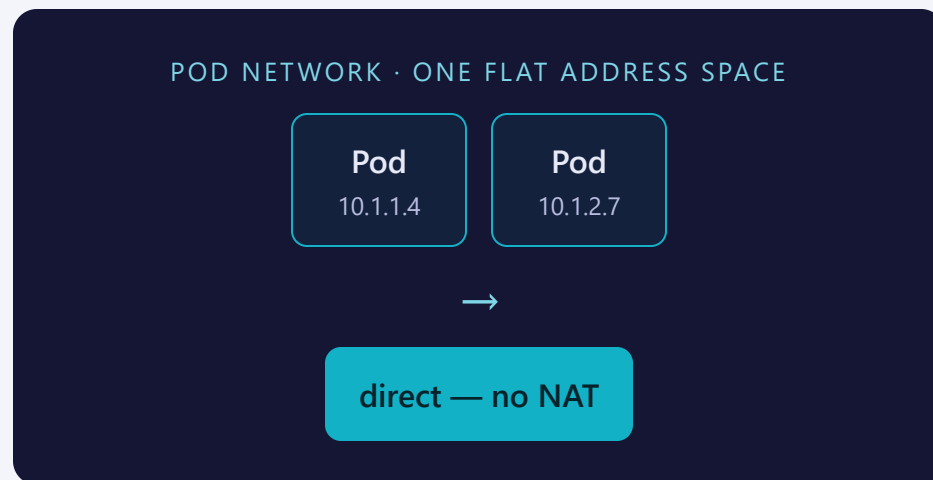
- State the flat-network guarantee in one sentence
- Curl a Pod directly by its IP
- Show that a Pod IP doesn't survive a recreate
- Explain why that motivates Services

CONCEPT

Every Pod is a first-class citizen of the network

The moment a Pod is scheduled, it gets an IP from the cluster's pod network — automatically, with **nothing to configure** in the manifest.

That IP is real and routable: any other Pod, on any node, can dial it directly. This is provided by the cluster's **CNI plugin**, not by anything you write.



Same node or not, the path is identical

WEB-SERVER $\leftrightarrow$ TEST-CLIENT — DIRECT ROUTING ACROSS NODE BOUNDARIES

Node A

web-server

10.1.1.4



Node B

test-client

10.1.2.7

10.1.2.7 $\rightarrow$ 10.1.1.4 direct — same address space, no translation, no Service required

No networking config — because none is needed

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server
  namespace: training
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

- No IP, no CIDR, no interface — **nothing to set** for networking
- containerPort just documents the listening port; it doesn't open one

WATCH

The Pod IP is assigned by the cluster at schedule time. You never write it — you only look it up.

Run it fast, or declare it

IMPERATIVE

```
$ kubectl run web-server \  
  --image=nginx:1.25 --labels=app=web \  
  --port=80 -n training
```

DECLARATIVE

```
$ kubectl apply -f pod-a.yaml
```

THE BRIDGE TRICK

```
$ kubectl run web-server --image=nginx:1.25 --port=80 -n training \  
  --dry-run=client -o yaml > pod-a.yaml
```

SEE IT

Curl the Pod IP — no Service in sight

Look up the IP, then hit it directly from another Pod. No DNS name, no virtual IP — just the raw address the cluster handed out.

It works from `test-client` regardless of which node either Pod landed on.

```
$ kubectl get pods -n training -o wide
NAME          READY  STATUS   IP          NODE
web-server    1/1    Running  10.1.1.4    node-1
test-client   1/1    Running  10.1.2.7    node-2

$ kubectl exec test-client -n training -- \
    wget -qO- $(kubectl get pod web-server -n
training \
    -o jsonpath='{.status.podIP}')
<!DOCTYPE html> ... Welcome to nginx!
```

THE CATCH

A Pod's IP only lives as long as the Pod does

SAME MANIFEST, TWO DIFFERENT IPS

web-server created
10.1.1.4



deleted



web-server recreated
10.1.1.9 — new IP

Delete and reapply the exact same `pod-a.yaml` and the new Pod comes back with a **different IP**. Recreation always gets a fresh address.

NEVER

Hardcode a Pod IP in application config — it will not survive a restart.

Where people trip

- **Treating a Pod IP as durable.** Even an unchanged manifest gets a new IP on every recreate.
- **No Service, no DNS name.** A raw Pod IP is fine for a quick test — not for anything that needs to survive a restart.
- **Curling the wrong port.** Hit the container's actual listening port, not an assumed one.
- **Blaming Kubernetes for a CNI problem.** Cross-node connectivity failures are almost always the CNI plugin, not the API.

Commands to keep at hand

```
$ kubectl get pods -n NS -o wide           # IP + node per Pod
$ kubectl get pod NAME -n NS -o jsonpath='{.status.podIP}' # just the IP
$ kubectl exec CLIENT -n NS -- wget -qO- POD_IP           # test by raw IP
$ kubectl delete pod NAME -n NS --force --grace-period=0   # recreate, watch the IP change
```

Tip: if a straight IP curl fails cross-node, suspect the CNI before you suspect your YAML.

RECAP

Pod-to-Pod connectivity in one breath

- Every Pod gets a routable IP from the pod network — automatic, nothing to configure
- Pod-to-Pod traffic is direct, no NAT, regardless of node
- Pod IPs are ephemeral — recreate a Pod, get a new IP
- Never hardcode a Pod IP — that's exactly the problem Services solve

HANDS-ON

Now run Lab 4.3
[labs/4.3/](#)

Next: [4.4 — Endpoints & Headless Services](#)